Machine-Checked Proof Gaps in the FALCON-512 EUF-CMA Reduction

A Lean 4 Formalization of Explicit Concrete-Security Losses

Working Draft

April 4, 2026

Abstract

We present a Lean 4 formalization of the FALCON-512 EUF-CMA security reduction that makes every loss term in the reduction explicit and machine-checks twelve proof gaps without `sorry` under Mathlib4 v4.28.0. Our main theorem, `falcon_security_reduction`, exposes the reduction bound

$$\varepsilon_{\text{forge}} \leq Q_H \varepsilon_{\text{SIS}} + \delta_{\text{rej}} + \delta_{\text{fp}} + \delta_{\text{smooth}} + \delta_{\text{hash}} + Q_S \delta_{\text{abort}},$$

so that approximation losses, oracle-programming losses, and implementation-sensitive terms appear as named objects rather than as hidden prose assumptions. We classify all twelve machine-checked gaps by severity and identify two as decisive for concrete security interpretation: GAP 7, where a naive Rényi-to-total-variation conversion yields an approximately 32-bit degradation path, and GAP 10, where random-oracle programming contributes a Q_H loss that pushes the underlying NTRU-SIS hardness target to roughly 192 bits for 128-bit forgery security in the stated model. We also report a supporting surrogate-model stress test based on a noisy quantum-sampler thought experiment, but keep that result explicitly secondary: it does not construct a forgery attack and does not imply that deployed Falcon implementations are broken. The contribution of this paper is therefore not a new exploit, but a machine-checked accounting of where concrete security is lost, what assumptions must stay visible, and which parts of the FALCON proof story are genuinely load-bearing.

1 Introduction

Security reductions for lattice signatures often distribute the real loss story across multiple proof ingredients: Gaussian-sampling arguments, rejection-sampling lemmas, smoothing-parameter bounds, random-oracle-programming steps, and implementation-sensitive approximations. For FALCON-512, that proof story is sophisticated enough that reader-facing summaries can easily obscure which losses are explicit, which are mitigated, and which are merely controlled heuristically.

This draft takes a theorem-first approach. Instead of retelling the FALCON proof at the level of informal prose, we formalize its EUF-CMA reduction in Lean 4 and make the reduction losses explicit in a single machine-checked bound. The resulting formalization identifies twelve proof gaps, classifies them by severity, and isolates the two load-bearing concrete-security effects: the naive Rényi-to-total-variation conversion loss (GAP 7) and the random-oracle-programming loss (GAP 10).

The main point is not that FALCON is practically broken. It is not. The point is that theorem-level loss accounting matters, and that a machine-checked representation of the reduction exposes exactly where the concrete-security story depends on nontrivial proof choices. This includes the

need to reason carefully about the Rényi framework rather than collapsing everything into total variation, and the need to keep the Q_H loss explicit when translating reduction quality into required underlying hardness.

The repository also contains a supporting surrogate-model stress test based on an artificial noisy quantum-sampler model. We keep that material explicitly secondary. It provides a cautionary example of how quickly proof guarantees can become vacuous inside an artificial model, but it does not construct a forgery attack and it does not describe deployed Falcon implementations.

Contributions. This draft makes four contributions:

1. a Lean 4 formalization of the FALCON-512 EUF-CMA reduction with explicit loss terms;
2. a machine-checked taxonomy of twelve proof gaps;
3. a concrete-security interpretation centered on GAP 7 and GAP 10;
4. a source-disciplined framing of supporting surrogate-model evidence that preserves the explicit no-attack boundary.

Positioning. This paper sits between cryptographic proof analysis and proof-assistant formalization. On the cryptographic side, we stay close to the Falcon specification and to the Rényi-divergence proof machinery used to interpret GAP 7 [1, 3, 4]. On the formal-methods side, the contribution is a Lean 4 / Mathlib artifact that turns the reduction's load-bearing approximations into named objects that can be cited and audited [8, 9]. The result is not an end-to-end implementation verification and it is not an attack paper. It is a theorem-level loss audit of the published reduction, written first for ePrint circulation and only secondarily as a seed for a later CRYPTO / Eurocrypt submission.

2 The Reduction and Its Lean Encoding

The formal development is organized around the theorem `falcon_security_reduction` in `artifacts/aristotle-p` It packages the reduction from EUF-CMA forgery to underlying NTRU-SIS hardness while exposing every intermediate loss term as a named field of the structure `ReductionErrors`. The resulting bound is

$$\varepsilon_{\text{forge}} \leq Q_H \varepsilon_{\text{SIS}} + \delta_{\text{rej}} + \delta_{\text{fp}} + \delta_{\text{smooth}} + \delta_{\text{hash}} + Q_S \delta_{\text{abort}}. \tag{1}$$

Equation (1) is the center of the manuscript. It is where the proof stops being a single headline claim and becomes an explicit accounting identity. The Q_H term captures the random-oracle-programming loss, the Q_S term captures signing-abort amplification, and the remaining delta terms isolate the parts of the proof that depend on rejection-sampling behavior, floating-point Gaussian approximation, smoothing-parameter control, and hash-to-point behavior.

The Lean formalization is not a full end-to-end proof of FN-DSA security, and we do not claim otherwise. Its contribution is narrower and, for this paper, more useful: every load-bearing approximation or bounding step becomes explicit enough to classify, cite, and interpret. In particular, the development builds a canonical vocabulary for discussing where concrete security is lost and what must be assumed for the published 128-bit story to hold in the intended proof model.

Two source-level facts matter especially for the manuscript:

1. the theorem and its explicit loss decomposition must remain recognizable in reader-facing prose; and

Manuscript object	Lean anchor	Repo source
Main reduction bound	`falcon_security_reduction`	`SecurityReduction.lean`
Dominant error term	`falcon512_dominant_error`	`SecurityReduction.lean`
GAP 7 discussion	`gap7_renyi_to_advantage_loss`	`RejectionSampling.lean`
GAP 10 discussion	`gap10_rom_programming_loss`	`RandomOracle.lean`

Table 1: Core theorem-to-source mapping for the load-bearing manuscript claims. All files live under `artifacts/aristotle-proof-gap-analysis/RequestProject/Falcon/`.

2. the surrounding discussion must preserve the distinction between a machine-checked formal gap analysis and an actual attack construction.

3 Twelve Machine-Checked Proof Gaps

The accompanying formal gap report identifies twelve proof gaps and groups them into four severity classes: critical, moderate, minor, and mitigated. The goal of the taxonomy is not to claim that every gap is equally dangerous; it is to prevent the reduction story from collapsing into an undifferentiated security headline.

The two critical gaps are GAP 7 and GAP 10. GAP 7 concerns the loss incurred by a naive Rényi-to-total-variation conversion. GAP 10 concerns the cost of random-oracle programming, including the need to guess the decisive hash query and, in related settings, absorb additional forking-style losses. These are the gaps with the clearest concrete-security effect on parameter selection.

The moderate gaps capture places where the proof remains sensitive to key quality, smoothing assumptions, or rejection-sampling behavior. The minor gaps record quantitatively smaller but still explicit mismatches between the idealized argument and a concrete implementation story. The mitigated gaps remain in the table because they are part of the full machine-checked account, but the surrounding proof or standardization story already contains plausible mechanisms for containing them.

The point of retaining all twelve gaps in the paper is methodological as much as technical. A formal proof-gap analysis is only credible if it shows the entire taxonomy rather than spotlighting one or two striking items while hiding the rest of the ledger.

4 Concrete-Security Interpretation

GAP 7: Rényi-to-total-variation loss

The Lean theorem `gap7_renyi_to_advantage_loss` records a basic but consequential fact: a naive conversion from order-2 Rényi divergence to total variation introduces a square-root loss. In small-parameter language, an ε-scale Rényi quantity becomes an approximately $\sqrt{\varepsilon}$-scale total-variation quantity. For the concrete scale discussed in the formal gap report, this turns a 2^{-64} quantity into an approximately 2^{-32} bound. The companion Lean theorem `falcon512_dominant_error`, also in `artifacts/aristotle-proof-gap-analysis/RequestProject/Falcon/SecurityReduction.lean`, records the corresponding dominance statement for the concrete FALCON-512 error instantiation: the $\delta_{\mathrm{rej}} \approx 2^{-32}$ term dominates δ_{fp}, δ_{smooth}, and δ_{hash}.

This is the source of the headline "32-bit degradation" language, but it needs to be stated with care. The published FALCON proof does not simply throw away the Rényi structure and convert everything to total variation at the first opportunity. Instead, it relies on maintaining the Rényi

framework through the reduction. The machine-checked point is therefore conditional and useful: if one reasons naively at the total-variation level, the loss is brutal; if one wants the stronger story, the Rényi chain must be carried carefully and explicitly through the entire proof pipeline.

GAP 10: random-oracle programming loss

The Lean theorem `gap10_rom_programming_loss` captures the random-oracle-programming cost. The reduction must guess which of the adversary's hash queries will become decisive, and that guess costs a factor of Q_H. In the concrete-security discussion this matters immediately: if one wants 128-bit forgery security in the stated model, the underlying NTRU-SIS hardness target must absorb that Q_H loss. The formal gap report summarizes this as a roughly 192-bit hardness requirement.

In that sense, GAP 10 is less subtle than GAP 7 and more operational. It says that even before one worries about secondary approximation losses, the reduction's dependence on query complexity already pushes the underlying hardness target beyond a naive "128-bit in, 128-bit out" reading. The theorem-first paper should make this explicit rather than letting it hide in reduction folklore.

Interplay

The most important concrete-security lesson is the interaction of GAP 7 and GAP 10. GAP 10 already taxes the reduction through Q_H. GAP 7 then punishes any naive attempt to convert the proof story into total-variation language. Together they explain why the machine-checked formalization is useful: the formal artifact makes visible exactly which assumptions must remain live if one wants to preserve the strongest reader-facing security interpretation.

5 Supporting Surrogate-Model Stress Test

The repository also contains a separate Lean development for a noisy quantum-sampler thought experiment. Inside that artificial model, the reduction can become vacuous at noise levels far above the tiny threshold required by the reduction bound. The correct statement is therefore about proof vacuity inside a surrogate model, not about a practical attack on deployed Falcon. No forgery attack is constructed, and the surrogate summary is explicit that correctness fails first in the noisy-sampler story: sufficiently noisy outputs become invalid signatures before they become useful attack tools. We keep this material brief because it is supporting context, not the paper's headline result.

6 Conclusion

The contribution of this draft is a machine-checked loss ledger for the FALCON-512 EUF-CMA reduction. By formalizing the reduction in Lean 4 and exposing the theorem-level loss structure directly, the project turns informal proof folklore into explicit objects that can be classified, cited, and scrutinized. The resulting taxonomy contains twelve proof gaps, but two deserve special emphasis for concrete security interpretation: GAP 7, which punishes a naive Rényi-to-total-variation reading, and GAP 10, which keeps the Q_H loss visible in the random-oracle step.

This is not a paper about breaking Falcon in practice. It is a paper about where a reduction loses concrete security, how those losses compose in a theorem-first representation, and why machine-checked formalisms are a useful tool for keeping that story honest. Natural follow-up directions include tightening the formal treatment of the Rényi chain, extending the formalization to a broader portion of the lattice-signature proof stack, and deciding whether the supporting surrogate-model stress test deserves a separate note or a later appendix-focused expansion.

References

[1] Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. *Falcon: Fast-Fourier Lattice-Based Compact Signatures over NTRU*. Specification v1.2, 2020.

[2] National Institute of Standards and Technology. *Post-Quantum Cryptography*. Computer Security Resource Center project page, updated December 11, 2025. `https://csrc.nist.gov/projects/post-quantum-cryptography`.

[3] Shi Bai, Damien Stehlé, and Weiqiang Wen. Improved security proofs in lattice-based cryptography: Using the Rényi divergence rather than the statistical distance. ASIACRYPT 2015.

[4] Thomas Prest. Sharper bounds in lattice-based cryptography using the Rényi divergence. ASIACRYPT 2017.

[5] Lov K. Grover. A Fast Quantum Mechanical Algorithm for Database Search. STOC 1996.

[6] Martin Roetteler, Michael Naehrig, Krysta M. Svore, and Kristen Lauter. *Quantum Resource Estimates for Computing Elliptic Curve Discrete Logarithms*. IACR ePrint 2017/598, 2017. `https://eprint.iacr.org/2017/598`.

[7] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. `https://doi.org/10.22331/q-2021-04-15-433`.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 Theorem Prover and Programming Language. CADE 2021.

[9] The mathlib Community. *mathlib4: The Lean Mathematical Library*. `https://github.com/leanprover-community/mathlib4`.

Gap	Issue	Severity	Primary anchor	Concrete note
1	Gram-Schmidt norm bound	Moderate	`edge_case1_unbalanced_key` (`EdgeCases.lean`)	Security argument depends on a high-probability bound rather than a uniform guarantee over all valid keys.
2	Floating-point sampling correlation	Minor	`gap2_fp_composition` `edge_case2_error_propagation`	Sampling introduces correlation beyond the simplest independent-error story.
3	Smoothing-parameter dependence on λ_1	Moderate	`gap3_smoothing_requirement` `edge_case7_ntru_vs_missing`	Tightness depends on a lattice minimum-distance assumption not proved directly for all relevant NTRU instances.
4	Key-dependent signing probability	Minor	`gap4_signing_prob_key_dependent`	Acceptance probability varies with key quality, so some valid keys are weaker than the average-case picture suggests.
5	CDT precision table	Mitigated	`gap5_cdt_subsumed`	Included for completeness; formally shown to be subsumed at the stated precision.
6	Center-dependent rejection rate	Moderate	`gap6_center_dependent`	Fixed M rejection bound M is not uniformly valid for all center offsets.
7	Rényi-to-TV square-root loss	Critical	`gap7_renyi_to_advantage_loss`	A generic conversion from Rényi divergence to total variation turns a 2^{-64}-scale quantity into an approximately 2^{-32} bound.
8	Timing side-channel leakage	Mitigated	`gap8_timing_distinguisher`	Rejection-count variability is distinguishable in principle, but constant-time implementations mitigate the issue in practice.
9	Hash-to-point bias	Mitigated	`gap9_per_coefficient_bias`	Bias exists without rejection, but the standardized hash-to-point procedure removes it.
10	Random-oracle programming loss	Critical	`gap10_rom_programming_loss` `gap10_forking_lemma_loss`	The reduction loses a factor of Q_H, pushing the underlying hardness target to roughly 192 bits for 128-bit forgery security in the stated model.
11	Multi-target loss accounting	Mitigated	`gap11_multi_target_loss` `edge_case6_multi_target`	The tighter reduction avoids the naive $Q_H Q_S$ loss.
12	SHAKE-256 algebraic structure	Minor	`shake256_collision_resistance`	No generic attack is known, but the ROM-to-instantiation boundary remains explicit.

Table 2: All twelve machine-checked proof gaps identified in the Lean 4 formalization of the FALCON-512 EUF-CMA reduction. GAP 7 and GAP 10 are the dominant concrete-security findings.